МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра інженерії програмного забезпечення

# КУРСОВА РОБОТА

(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни: «Об’єктно-орієнтоване програмування»

на тему:

**«Система контролю часу та планування задач»**

студента ІІІ курсу групи ІПЗ-23-4
спеціальності 121 «Інженерія програмного забезпечення»
_____Отовчиць Ілля Сергійович_____
(прізвище, ім’я та по-батькові)

Керівник: старший викладач кафедри ІПЗ Олег ВЛАСЕНКО

Дата захисту: " ___ " _____ 20__ р.
Національна шкала _____
Кількість балів: _____
Оцінка: ECTS _____

Члени комісії

_____ Тетяна ВАКАЛЮК
(підпис) (прізвище та ініціали)

_____ Олексій ЧИЖМОТРЯ
(підпис) (прізвище та ініціали)

_____ Олег ВЛАСЕНКО
(підпис) (прізвище та ініціали)

_____ Дмитро ПРОХОРЧУК
(підпис) (прізвище та ініціали)

Житомир – 2026

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Освітній рівень: бакалавр
Спеціальність 121 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»
Завідувач кафедри ІПЗ
_____ Тетяна ВАКАЛЮК
«____»_____2026 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Отовчицю Іллі Сергійовичу

1. Тема роботи: Система контролю часу та планування задач,
керівник роботи: ст. викладач кафедри ІПЗ ВЛАСЕНКО Олег
2. Строк подання студентом: « 09. » травня 2026 р.
3. Вихідні дані до роботи: Розробити програму для системи
4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)

1. Постановка завдання
2. Аналіз аналогічних розробок
3. Алгоритми роботи програми
4. Опис роботи програми
5. Програмне дослідження

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

1. Презентація до КР
2. Посилання на репозиторій: https://github.com/shino-bit/Time_control_and_task_planning_system

6. Консультанти розділів роботи

| Розділ | Прізвище, ініціали та посади консультанта | Підпис, дата | |
|---|---|---|---|
| | | завдання видав | завдання прийняв |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

7. Дата видачі завдання " 01 " березня 2024 р.

**КАЛЕНДАРНИЙ ПЛАН**

| № з/п | Назва етапів курсового проєктування | Строк виконання етапів роботи | Примітки |
|---|---|---|---|
| 1 | Постановка задачі | 15.02.2026 | виконано |
| 2 | Пошук, огляд та аналіз аналогічних розробок | 16.02.2026-18.02.2026 | виконано |
| 3 | Формулювання технічного завдання | 18.02.2026-18.02.2026 | виконано |
| 4 | Опрацювання літературних джерел | 19.02.2026-21.02.2026 | виконано |
| 5 | Проектування структури | 22.02.2026-26.02.2026 | виконано |
| 6 | Написання програмного коду | 02.03.2026-04.03.2026 | виконано |
| 7 | Відлагодження | 05.03.2026 | виконано |
| 8 | Написання пояснювальної записки | 06.03.2026-09.03.2026 | виконано |
| 9 | Захист | | |

**Студент** _____ Ілля ОТОВЧИЦЬ
(підпис) (прізвище та ініціали)

**Керівник роботи** _____ Олег ВЛАСЕНКО
(підпис) (прізвище та ініціали)

# РЕФЕРАТ

Завданням курсової роботи було розроблення системи контролю часу та планування задач.

Пояснювальна записка до курсової роботи на тему «Система контролю часу та планування задач» складається з переліку умовних скорочень та словника, вступу, трьох розділів, висновків, списку використаної літератури та додатків.

Текстова частина викладена на 30 сторінках друкованого тексту.

Список використаних джерел містить 10 найменувань і займає 1 сторінку. У роботі наведено 12 рисунків. Пояснювальна записка має 5 сторінок додатків. Загальний обсяг роботи — 36 сторінок.

Ключові слова: WPF, MVVM, VIEWMODEL, C#, ООП, ТАЙМ-МЕНЕДЖМЕНТ, ПЛАНУВАННЯ, ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ.

Зміст

РЕФЕРАТ ..... 4
ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ, СЛОВНИК ..... 6
ВСТУП ..... 7
РОЗДІЛ 1 АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ ..... 8
1.1 Аналіз задачі, засобів та методів її вирішення ..... 8
1.2 Аналіз існуючого програмного забезпечення за тематикою курсової роботи: Система контролю часу та планування задач ..... 9
РОЗДІЛ 2 ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ..... 13
2.1. Проектування загального алгоритму роботи програми ..... 13
Навігація та відображення станів ..... 13
2.2. Розробка функціональних алгоритмів роботи програми ..... 16
2.3. Реалізація механізму оновлення інтерфейсу ..... 18
Розробка основного функціоналу та бізнес-логіки ..... 21
Логіка таймера та відліку часу ..... 22
Алгоритми сортування та фільтрації ..... 23
РОЗДІЛ 3 ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ ..... 25
3.1 Опис роботи з програмним додатком (Опис інтерфейсу) ..... 25
3.2 Тестування роботи програмного забезпечення ..... 27
ВИСНОВКИ ..... 30
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ ..... 32

# ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ, СЛОВНИК

Рис. – рисунок

ПЗ – програмне забезпечення

ООП – об'єктно-орієнтоване програмування

WPF – Windows Presentation Foundation

MVVM – Model-View-ViewModel

JSON – JavaScript Object Notation

UI – інтерфейс користувача (User Interface)

# ВСТУП

Актуальність теми. Тема розробки програмного забезпечення для планування та контролю часу є актуальною в контексті сучасного темпу життя, де ефективне управління власними ресурсами стає ключовим фактором успіху. Зростання обсягів інформації та постійна багатозадачність вимагають надійних інструментів для організації особистих і робочих справ.

Створення власної системи тайм-менеджменту дозволяє не лише автоматизувати процес планування, але й адаптувати його під індивідуальні потреби користувача, використовуючи новітні технології та програмні засоби. Це відкриває широкі можливості для впровадження корисних функцій, які часто є платними або відсутніми у стандартних аналогах.

Метою створення даного курсового проєкту є розробка десктопного додатку «Система контролю часу та планування задач» для комп'ютерної платформи, що поєднує в собі функції списку справ (To-Do) та трекера часу. Основна увага приділяється розробці зручного користувацького інтерфейсу, функціоналу для сортування завдань за терміновістю та алгоритмам, що забезпечать надійне збереження даних.

Об’єктом дослідження є процес організації особистого часу та методи автоматизації планування завдань і моніторингу їх виконання.

Предметом дослідження є можливості платформи Windows Presentation Foundation (WPF) та архітектурного патерну MVVM для розробки додатків, а також особливості об'єктно-орієнтованого програмування (ООП) у контексті створення систем обліку часу. Вивчення цих аспектів дозволить розробити якісний програмний продукт, використовуючи сучасні технології для підвищення його надійності, швидкодії та загальної функціональності.

# РОЗДІЛ 1 АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ

## 1.1 Аналіз задачі, засобів та методів її вирішення

Основним завданням є розробка програмної системи контролю часу та планування задач, яка гармонійно поєднує функціонал списку справ To-Do List із вбудованим трекером часу.

Після аналізу поставленої задачі та різних підходів до створення десктопних додатків, було вирішено, що найкращою графічною підсистемою для реалізації проєкту є платформа WPF (Windows Presentation Foundation). Ця технологія надає широкі можливості для створення сучасних, функціональних та інтерактивних користувацьких інтерфейсів.

Завдяки потужному механізму прив'язки даних (Data Binding), використання архітектурного патерну MVVM (Model-View-ViewModel) є найбільш оптимальним вибором. Патерн MVVM дозволяє ефективно відокремити візуальний інтерфейс користувача від бізнес-логіки програми (наприклад, логіки відліку часу та алгоритмів сортування). Це суттєво полегшує розробку, тестування та подальшу підтримку програми. Таке рішення також сприяє гнучкості та масштабованості коду, оскільки будь-які зміни в даних (наприклад, зміна статусу задачі або оновлення лічильника секунд) миттєво та автоматично відображаються на інтерфейсі користувача.

Крім того, використання WPF у поєднанні з MVVM дозволяє реалізувати інтуїтивно зрозумілу та зручну взаємодію. Для додатків категорії тайм-менеджменту швидкість і зручність додавання нових задач є критично важливими для забезпечення позитивного користувацького досвіду та підвищення продуктивності.

Щоб зрозуміти напрямок подальшого розвитку проєкту, потрібно проаналізувати існуючі програмні забезпечення, зокрема популярні планувальники та трекери часу, які використовують подібні принципи. Це допоможе виявити найкращі практики проєктування інтерфейсів та уникнути поширених помилок, таких як перевантаженість візуального простору або занадто складна навігація.

Аналіз існуючих рішень включає вивчення методів категоризації завдань, підходів до візуалізації дедлайнів, термінів виконання, способів реалізації таймерів, а також механізмів локального збереження даних користувача зокрема, використання легковагового формату JSON замість громіздких баз даних. Це допоможе створити комплексний та сучасний продукт, який буде повністю відповідати очікуванням користувачів, що прагнуть ефективно організувати свій час.

## 1.2 Аналіз існуючого програмного забезпечення за тематикою курсової роботи: Система контролю часу та планування задач

**Програмне забезпечення “Microsoft To Do”**

В першу чергу потрібно проаналізувати та оцінити функціонал популярних базових планувальників, яскравим представником яких є додаток Microsoft To Do.

Головне меню програми:

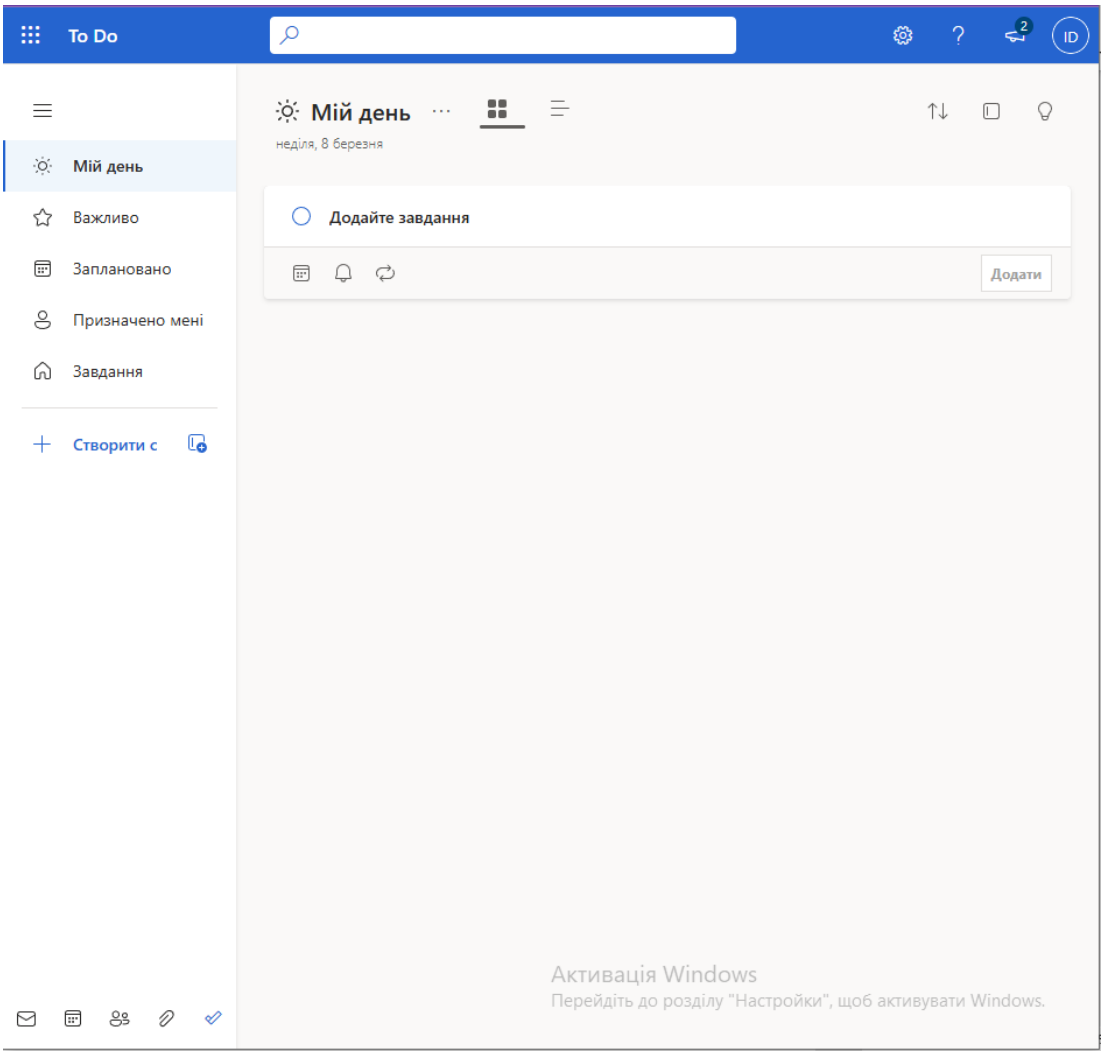

Рис. 1.1. Інтерфейс програми “Microsoft To Do”

На рисунку 1.1 можна побачити основні елементи управління:

1. Списки задач: Мій день, Важливо, Заплановано.
2. Поле для швидкого додавання нового завдання.
3. Перелік створених задач із можливістю відмітити їх як виконані.

Помітно, що інтерфейс є мінімалістичним та інтуїтивно зрозумілим. При виборі конкретної задачі відкривається бічна панель, де можна встановити дедлайн, термін виконання або додати нагадування.

Проте, головним недоліком цієї системи в контексті нашої задачі є повна відсутність інструментів для обліку часу. Користувач може лише констатувати факт виконання поставивши галочку, але не має змоги відстежити, скільки саме хвилин чи годин було витрачено на роботу.

**Програмне забезпечення “Toggl Track”**

Розглянемо інший підхід на прикладі професійного сервісу для обліку часу “Toggl Track”. Основний акцент тут зроблено саме на секундомірі та аналітиці.

Головне вікно містить такі розділи: таймер, звіти, проєкти та налаштування.

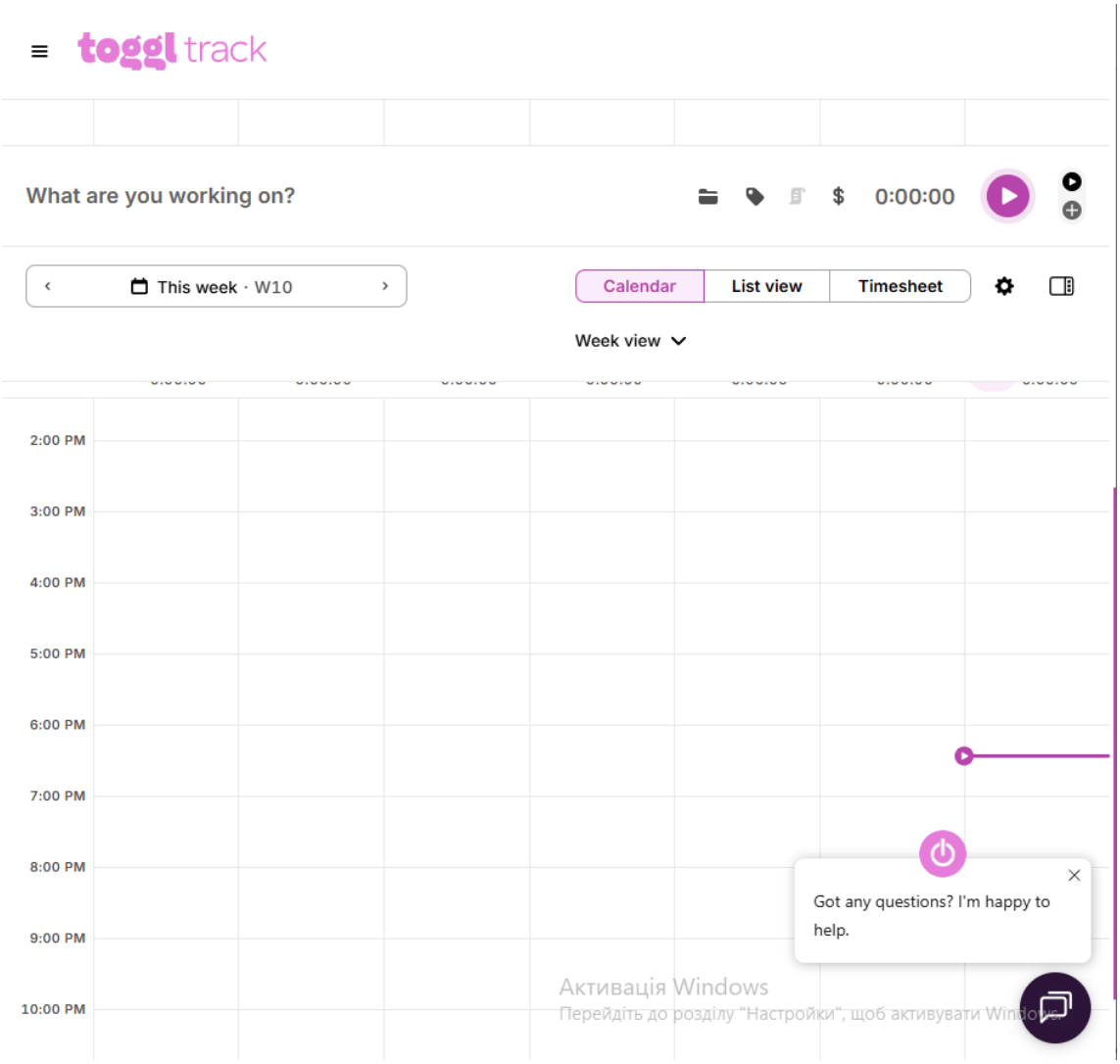


Рис. 1.2. Головне меню програми “Toggl Track”

У додатку наявний широкий спектр можливостей: • Таймер: запуск відліку часу для конкретної активності в один клік. • Статистика: детальні графіки витраченого часу за днями або тижнями. • Збереження: прогрес автоматично синхронізується з хмарним сховищем, що дозволяє продовжувати роботу на різних пристроях.

Незважаючи на потужний функціонал таймера, ця програма має слабку систему планування To-Do. Задачі тут виступають радше як "теги" для списання часу, а не як список справ на день із чіткими пріоритетами та дедлайнами. Крім того, розширені налаштування та детальна статистика доступні лише за платною підпискою.

Отже, існуючі рішення часто пропонують або лише планування, або лише трекінг часу, змушуючи користувача використовувати кілька програм одночасно.

Використання об'єднаного функціоналу в моєму проєкті може суттєво підвищити його зручність для користувачів. Цей підхід дозволить створити інструмент, який закриватиме обидві потреби: що потрібно зробити дедлайн і скільки часу це зайняло таймер.

Висновок: На основі проведеного аналізу я дійшов висновку, що в своїй курсовій роботі я зосереджусь на створенні гібридного десктопного додатку. Основна увага буде приділена впровадженню такого функціоналу: створення завдань із категоріями, вбудований секундомір для кожної задачі, зручне сортування та локальне збереження даних (без необхідності підключення до Інтернету). Це дозволить створити легку, безкоштовну та ефективну систему контролю часу.

# РОЗДІЛ 2 ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

## 2.1. Проектування загального алгоритму роботи програми

**Навігація та відображення станів**

В основі загальної роботи програми лежить гнучкий механізм керування станами та відображенням даних.

У додатку зміна інформації на екрані здійснюється через централізований механізм прив'язки даних (Data Binding), який використовує архітектурний патерн MVVM. Цей механізм забезпечує чисте розділення візуального представлення та бізнес-логіки, що суттєво полегшує підтримку та розширення коду в майбутньому.

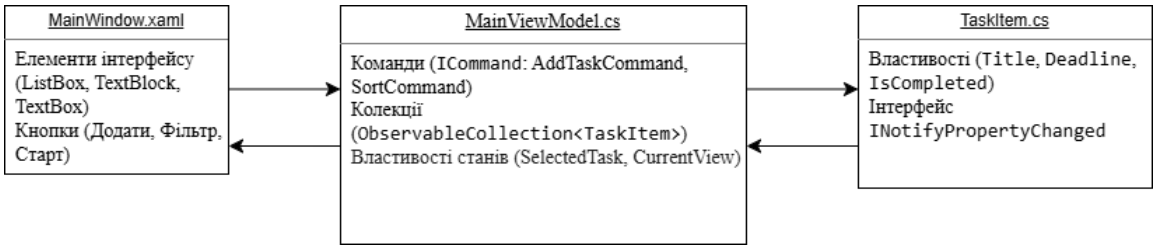


Рис. 2.1. Демонстрація механізму керування станами через MVVM

Головна модель представлення MainViewModel підписується на зміни в колекціях даних та станах інтерфейсу. Це означає, що кожного разу, коли користувач взаємодіє з програмою наприклад, перемикає вкладки або фільтри задач, MainViewModel дізнається про це і оновлює поточне відображення на екрані.

Наприклад, коли користувач натискає кнопку фільтрації “Термінові”, виконується відповідна команда наприклад, SortUrgentCommand. Коли ця команда виконується, вона викликає метод у MainViewModel для зміни параметрів CollectionViewSource, внаслідок чого інформує інтерфейс про необхідність перемалювати список задач у новому порядку.

MainViewModel отримує сповіщення про зміну стану, оновлює свою властивість, яка визначає поточний список для відображення, і повідомляє користувацький інтерфейс про необхідність оновлення через реалізацію інтерфейсу INotifyPropertyChanged.

Цей механізм керування станами забезпечує чітку і добре структуровану архітектуру програми, де кожен компонент виконує свою роль. Використання патерну MVVM для управління інтерфейсом дозволяє легко змінювати логіку сортування та відображення без прямого втручання у XML-код. Це робить програмний продукт більш підтримуваним і зручним для масштабування.

**Керування задачами та таймером**

Керування всіма подіями у системі здійснює головна модель представлення MainViewModel. Вона містить у собі методи ініціалізації базових колекцій, а також взаємодіє з окремими сервісами, які відповідають за збереження даних JsonDataService та відлік часу клас DispatcherTimer.

Рис. 2.2. Блок-схема роботи MainViewModel

На Рис. 2.2 зображено взаємозв’язки сервісів та моделі відображення у вигляді блок-схеми. ViewModel пов’язує моделі об’єктів задач TaskItem з їхнім виглядом в інтерфейсі. Після запуску програма здійснює завантаження збережених задач із JSON-файлу, виводячи їх на екран. Наступним кроком є ініціалізація внутрішніх підсистем: механізму таймера для відліку часу роботи над обраною задачею та команд для обробки дій користувача додавання, видалення, редагування.

Кожен з цих модулів відповідає за свій аспект функціонування програми та взаємодіє з MainViewModel, щоб забезпечити цілісність даних. Під час своєї роботи таймер та обробники подій сповіщають систему про зміни наприклад, зміна статусу виконання або оновлення лічильника секунд, надсилаючи відповідні тригери. Така взаємодія дозволяє додатку для тайм-менеджменту працювати швидко, безперебійно та інтерактивно.

### 2.2. Розробка функціональних алгоритмів роботи програми

Для початку спроєктуємо модель даних, яка буде основою нашого додатку. Головною сутністю є клас TaskItem (Рис. 2.3).

Ця модель включає набір характеристик властивостей, що описують конкретне завдання. Вона також реалізує інтерфейс INotifyPropertyChanged. Цей механізм є критично важливим: він спрацьовує при будь-якій зміні властивостей наприклад, коли таймер щосекунди збільшує значення витраченого часу та ініціює відповідну подію, що сигналізує інтерфейсу користувача (View) про необхідність оновлення відображення.

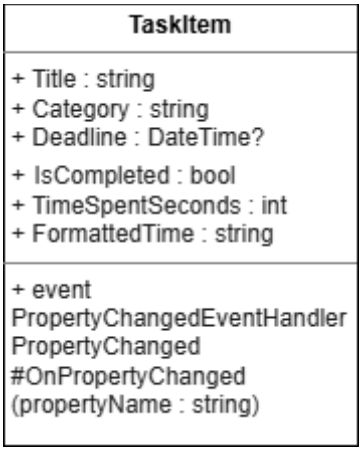


Рис. 2.3. Діаграма класу моделі об’єкта TaskItem

Така структуризація класів спрощує додавання нових властивостей до задач у майбутньому наприклад, описів або підзадач. Список цих об’єктів міститиметься у MainViewModel, що відповідатиме за логіку програми та зв’язок з відповідним відображенням головного вікна (Рис. 2.4).

Клас MainViewModel міститиме в собі взаємопов’язані об’єкти: колекцію задач ObservableCollection<TaskItem>, механізми фільтрації ICollectionView та системний таймер DispatcherTimer. Крім того, він буде містити всі необхідні команди ICommand для обробки дій користувача натискання кнопок додавання, старту, зупинки. Для забезпечення архітектурної чистоти, головна ViewModel може наслідувати BaseViewModel, яка бере на себе базову реалізацію інтерфейсу INotifyPropertyChanged.

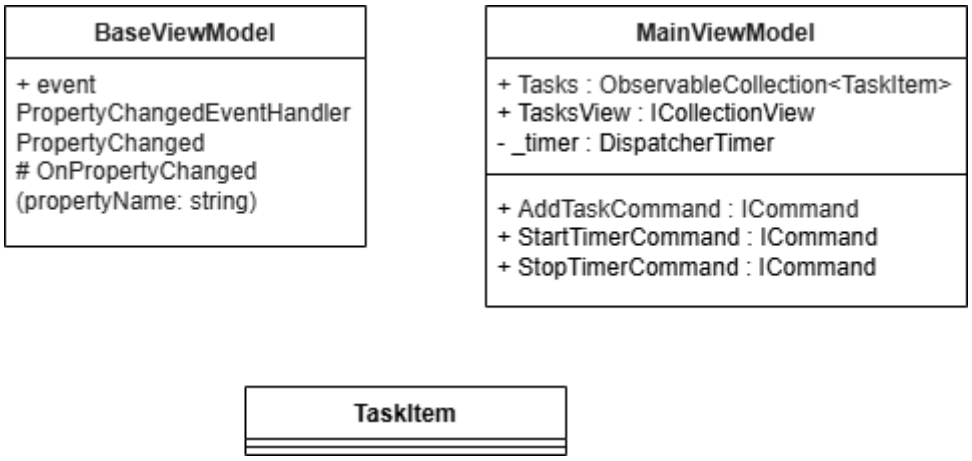


Рис. 2.4. Діаграма ієрархії та зв'язків ViewModel

Розглянемо впровадження бази збереження даних. Оскільки розроблюваний додаток є локальною десктопною програмою, використання повноцінних реляційних баз даних таких як, SQL Server є надлишковим і ускладнює розгортання програми. Тому було прийнято рішення використовувати легковаговий формат **JSON** для зберігання списку задач.

База даних (файл tasks.json) міститиме масив об'єктів із наступною структурою полів:

1. Title (string): Це назва задачі. Використання рядкового типу даних дозволяє зберігати текстову інформацію про мету завдання.
2. Category (string): Категорія задачі наприклад, "Робота", "Навчання". Дозволяє зручно групувати та фільтрувати завдання.
3. Deadline (datetime/null): Кінцевий термін виконання задачі. Використання формату дати-часу дозволяє системі автоматично

вираховувати, чи є задача простроченою. Це поле може бути порожнім (null), якщо задача не має жорсткого дедлайну.

4. IsCompleted (boolean): Логічний прапорець, що визначає статус виконання true — виконано, false — активно.
5. TimeSpentSeconds (int): Витрачений час. Використання цілочисельного типу даних (int) дозволяє ефективно зберігати накопичені секунди. Під час відображення в інтерфейсі це значення конвертується у зручний формат.

За серіалізацію запис у файл та десеріалізацію читання з файлу відповідає окремий сервісний клас JsonDataService.

Структура даних у форматі JSON утворює зв'язок "один до багатьох" на рівні колекції: один файл збереження містить багато незалежних задач. Структуру збереження даних зображено на Рис. 2.5.

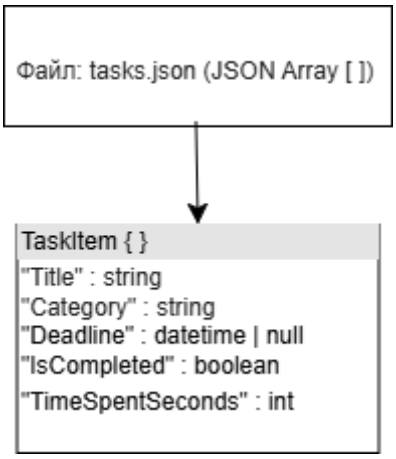


Рис. 2.5. Структура збереження даних JSON

### 2.3. Реалізація механізму оновлення інтерфейсу

Для забезпечення інтерактивності додатка та миттєвого відображення змін наприклад, ходу таймера або зміни статусу задачі було використано стандартний механізм сповіщень MVVM. Оскільки архітектура проєкту передбачає пряму взаємодію між властивостями об'єктів та елементами інтерфейсу, інтерфейс INotifyPropertyChanged було реалізовано безпосередньо у класі сутності TaskItem.

Основним елементом цього механізму є метод OnPropertyChanged, який ініціює подію при зміні будь-якого значення. Це дозволяє системі автоматично оновлювати інтерфейс користувача. Найбільш яскраво це проявляється під час роботи таймера: коли значення витраченого часу змінюється кожну секунду, це тригерує оновлення форматованого рядка на екрані:

```
public class TaskItem : INotifyPropertyChanged
{

    private int _timeSpentSeconds = 0;

    public int TimeSpentSeconds
    {
        get => _timeSpentSeconds;
        set
        {
            _timeSpentSeconds = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(FormattedTime));
        }
    }

    public string FormattedTime =>
TimeSpan.FromSeconds(TimeSpentSeconds).ToString(@"hh\:mm\:ss");


    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
   }
```

Наступним кроком є реалізація головної моделі представлення MainViewModel. Вона виконує роль координатора: тут відбувається завантаження збережених даних через JsonDataService, ініціалізація колекцій та налаштування таймера.

```
public class MainViewModel : INotifyPropertyChanged
{
    private readonly JsonDataService _dataService;
    private DispatcherTimer _timer;

    public ObservableCollection<TaskItem> Tasks { get; set; }
    public ObservableCollection<string> Categories { get; set; }


    public MainViewModel()
    {
        _dataService = new JsonDataService();
        var loadedTasks = _dataService.LoadTasks();
        Tasks = new ObservableCollection<TaskItem>(loadedTasks);
        Categories = new ObservableCollection<string> { "Загальне", "Навчання", "Робота",
"Особисте" };
        SelectedCategory = Categories.First();
        _timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(1) };
```

```
        _timer.Tick += Timer_Tick;
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
      }
```

У самій XAML-розмітці файлу MainWindow.xaml використовується механізм Binding для підключення елементів управління до колекцій та властивостей із DataContext. Наприклад, для відображення списку задач використовується елемент ListBox, який автоматично генерує візуальні елементи на основі шаблону DataTemplate:

```
<ListBox ItemsSource="{Binding TasksView}" SelectedItem="{Binding SelectedTask}"
         HorizontalContentAlignment="Stretch" BorderThickness="0"
Background="Transparent"
         ScrollViewer.HorizontalScrollBarVisibility="Disabled">

    <ListBox.ItemTemplate>
        <DataTemplate>
            <Border Background="White" Margin="0,0,0,5" Padding="10" CornerRadius="6"
BorderBrush="#E5E7EB" BorderThickness="1">
                <Grid>
                    <CheckBox Grid.Column="0" IsChecked="{Binding IsCompleted,
UpdateSourceTrigger=PropertyChanged}"
                              VerticalAlignment="Center" Margin="0,0,10,0"/>

                    <StackPanel Grid.Column="1" VerticalAlignment="Center">
                        <TextBlock Text="{Binding Title}" FontSize="14"
TextWrapping="Wrap" Margin="0,0,0,2">
                        </TextBlock>

                        <TextBlock Text="{Binding Deadline, StringFormat='Дедлайн:
{0:dd.MM.yyyy}'}" FontSize="11">
                        </TextBlock>
                    </StackPanel>

                    <Button Grid.Column="3" Content="×" Command="{Binding
DataContext.DeleteTaskCommand, RelativeSource={RelativeSource AncestorType=Window}}"
                             Background="Transparent" BorderThickness="0"
Foreground="#EF4444" FontWeight="Bold" Cursor="Hand" Margin="5,0,0,0"/>
                </Grid>
            </Border>
        </DataTemplate>
    </ListBox.ItemTemplate>
      </ListBox>
```

Цей архітектурний підхід допомагає забезпечити ефективне управління станами програми та відображенням різних частин інтерфейсу, зберігаючи при цьому чистоту, реактивність та високу структурованість коду згідно з патерном MVVM.

**Розробка основного функціоналу та бізнес-логіки**

Процес розробки ключового функціоналу системи контролю часу вимагає реалізації трьох основних підсистем: механізму збереження даних, логіки відліку часу та алгоритмів динамічного сортування. У даному підрозділі детально розглянуто реалізацію цих компонентів.

**Робота з локальними даними (JSON)**

Оскільки додаток призначений для персонального використання, інтеграція повноцінного SQL-сервера є недоцільною. Для збереження прогресу та списку задач було розроблено сервісний клас JsonDataService, який використовує вбудовану бібліотеку System.Text.Json. Це дозволяє швидко та безпечно серіалізувати об'єкти у текстовий файл.

```
public class JsonDataService
{
    private readonly string _filePath;

    public JsonDataService(string fileName = "tasks.json")
    {
        _filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, fileName);
    }

    public void SaveTasks(IEnumerable<TaskItem> tasks)
    {
        var options = new JsonSerializerOptions { WriteIndented = true };
        string json = JsonSerializer.Serialize(tasks, options);
        File.WriteAllText(_filePath, json);
    }

    public IEnumerable<TaskItem> LoadTasks()
    {
        if (!File.Exists(_filePath))
        {
            return new List<TaskItem>();
        }

        string json = File.ReadAllText(_filePath);
        return JsonSerializer.Deserialize<IEnumerable<TaskItem>>(json) ?? new
List<TaskItem>();
    }
  }
```

Метод SaveTasks перетворює колекцію об'єктів TaskItem у відформатований JSON-рядок і записує його у файл. Параметр WriteIndented = true робить збережений файл читабельним для людини. Метод LoadTasks перевіряє наявність файлу; якщо він існує, відбувається зворотний процес десеріалізації, інакше —

повертається порожній список. Такий підхід гарантує, що користувач не втратить свої задачі після закриття програми.

**Логіка таймера та відліку часу**

Ключовою фічею додатку є можливість відстежувати час виконання конкретної задачі. Для цього у класі MainViewModel реалізовано механізм на базі DispatcherTimer. На відміну від звичайних таймерів наприклад, System.Timers.Timer, DispatcherTimer працює в основному потоці інтерфейсу UI Thread, що дозволяє безпечно оновлювати візуальні елементи екрану без виникнення винятків конфлікту потоків.

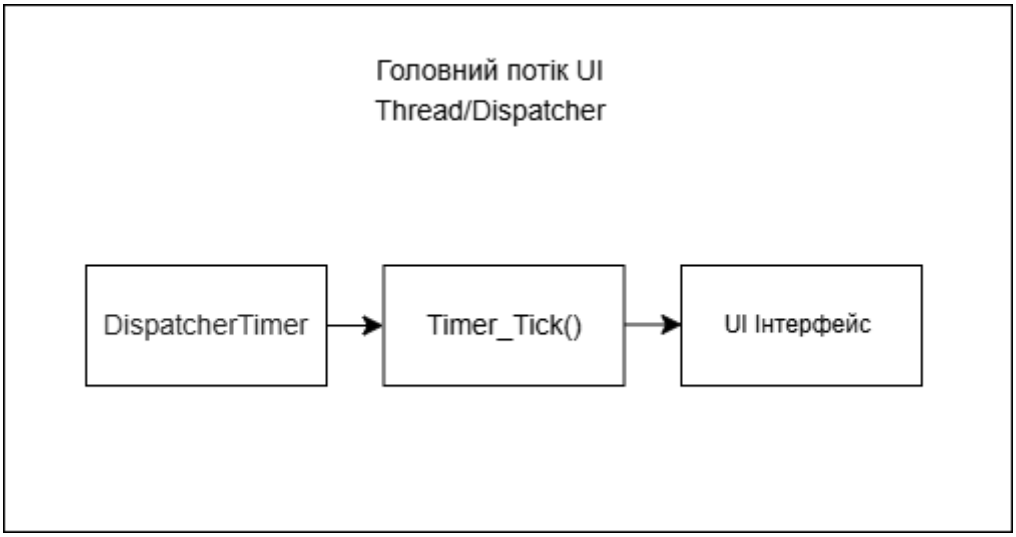


Рис. 2.6. Принцип роботи DispatcherTimer у головному потоці

```
public class MainViewModel : INotifyPropertyChanged
{
    private DispatcherTimer _timer;

    public MainViewModel()
    {
        _timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(1) };
        _timer.Tick += Timer_Tick;
    }

    private void Timer_Tick(object sender, EventArgs e)
    {
        if (SelectedTask != null)
        {
            SelectedTask.TimeSpentSeconds++;
            UpdateStatistics();
        }
    }
  }
```

Метод InitializeTimer задає інтервал спрацьовування в 1 секунду. При настанні події Tick програма перевіряє, чи обрана поточна задача та чи не є вона

вже виконаною. Якщо умови виконуються, лічильник секунд інкрементується. Завдяки реалізованому раніше інтерфейсу INotifyPropertyChanged, зміна властивості TimeSpentSeconds автоматично оновлює текст таймера на екрані користувача.

**Алгоритми сортування та фільтрації**

Для забезпечення ефективного планування, користувач повинен бачити найактуальніші завдання на початку списку. Для цього реалізовано алгоритм сортування "За терміновістю". Щоб не змінювати порядок елементів у самій колекції ObservableCollection що могло б вплинути на індексацію, сортування застосовується на рівні представлення View за допомогою CollectionViewSource.

```
public class MainViewModel : INotifyPropertyChanged
{
    public ICollectionView TasksView { get; }

    public MainViewModel()
    {
        TasksView = CollectionViewSource.GetDefaultView(Tasks);
        TasksView.SortDescriptions.Add(new SortDescription("IsCompleted",
ListSortDirection.Ascending));
        TasksView.SortDescriptions.Add(new SortDescription("Deadline",
ListSortDirection.Ascending));
    }

    private void ApplySort(object parameter)
    {
        TasksView.SortDescriptions.Clear();

        if (parameter is string sortType && sortType == "Urgency")
        {
            TasksView.SortDescriptions.Add(new SortDescription("IsCompleted",
ListSortDirection.Ascending));
            TasksView.SortDescriptions.Add(new SortDescription("Deadline",
ListSortDirection.Ascending));
        }
        else
        {
            TasksView.SortDescriptions.Add(new SortDescription("Category",
ListSortDirection.Ascending));
            TasksView.SortDescriptions.Add(new SortDescription("Title",
ListSortDirection.Ascending));
        }

        TasksView.Refresh();
    }

      }
```

Цей алгоритм працює каскадно. Клас SortDescription спочатку групує всі невиконані задачі (IsCompleted = false) у верхній частині списку. Далі, всередині цієї групи, задачі вишиковуються за зростанням дати Deadline. Таким чином,

прострочені та найближчі завдання автоматично опиняються на перших позиціях, що допомагає користувачеві сфокусуватися на найголовнішому.

**Висновок:** У процесі проектування та розробки системи було успішно застосовано патерн MVVM для чіткого розділення інтерфейсу та бізнес-логіки. Головна модель MainViewModel керує процесами завантаження та збереження даних через створений JsonDataService. Розроблено надійну логіку відліку часу за допомогою DispatcherTimer та впроваджено гнучкі алгоритми сортування через CollectionViewSource. Обраний підхід забезпечив створення швидкодіючого, стабільного та повністю автономного програмного продукту.

# РОЗДІЛ 3 ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ

## 3.1 Опис роботи з програмним додатком (Опис інтерфейсу)

Інтерфейс програми «Система контролю часу та планування задач» спроєктовано за принципом «одного вікна» Single-Window Interface. Це дозволяє користувачеві мати миттєвий доступ до всіх основних функцій без необхідності перемикатися між різними сторінками чи завантажувальними екранами, що є критично важливим для інструментів підвищення продуктивності.

Під час запуску програми користувача зустрічає головне робоче вікно, яке візуально поділене на кілька логічних та функціональних зон (Рис. 3.1).

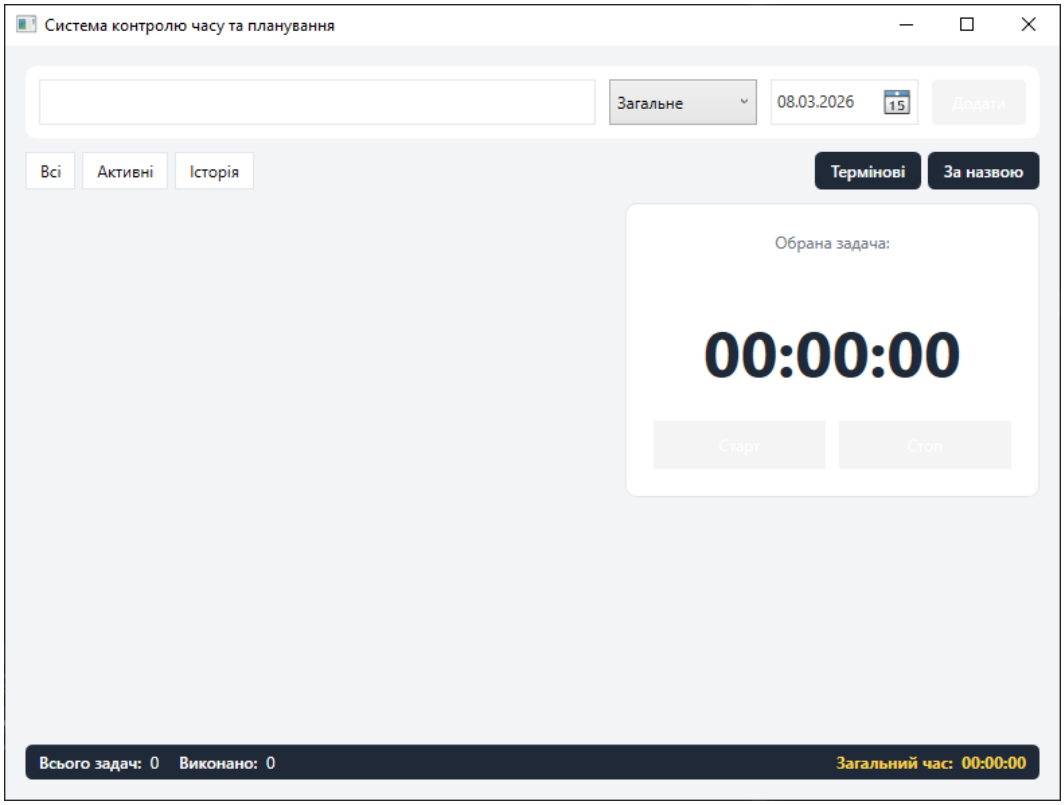


Рис. 3.1. Головне вікно програми

У верхній частині вікна розташована панель створення нових завдань. Для додавання задачі користувач має заповнити відповідні поля: ввести коротку назву в текстове поле, обрати категорію наприклад, «Робота» або «Навчання» з випадаючого списку та, за потреби, вказати дату дедлайну за допомогою

вбудованого календаря. Після заповнення даних натискається кнопка «Додати», і задача миттєво з'являється у системі.

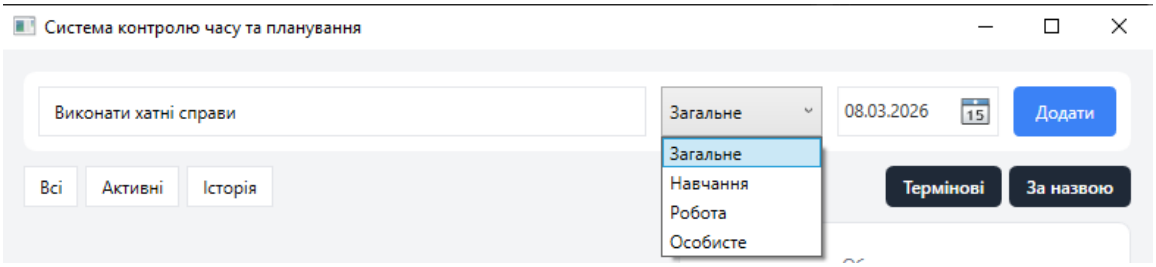


Рис. 3.2. Панель створення нової задачі

Центральну частину інтерфейсу займає динамічний список задач. Над списком розміщено зручну панель інструментів для фільтрації та сортування. Користувач може перемикати відображення між «Усіма», «Активними» та «Виконаними» задачами, а також застосовувати розумне сортування «За терміновістю» (Рис. 3.3).

Кожна задача в списку представлена у вигляді окремої картки, на якій відображається її назва, категорія, дедлайн та статус виконання у вигляді чекбоксу. Завдання, термін виконання яких вже минув, візуально виділяються наприклад, червоним кольором для негайного привернення уваги.

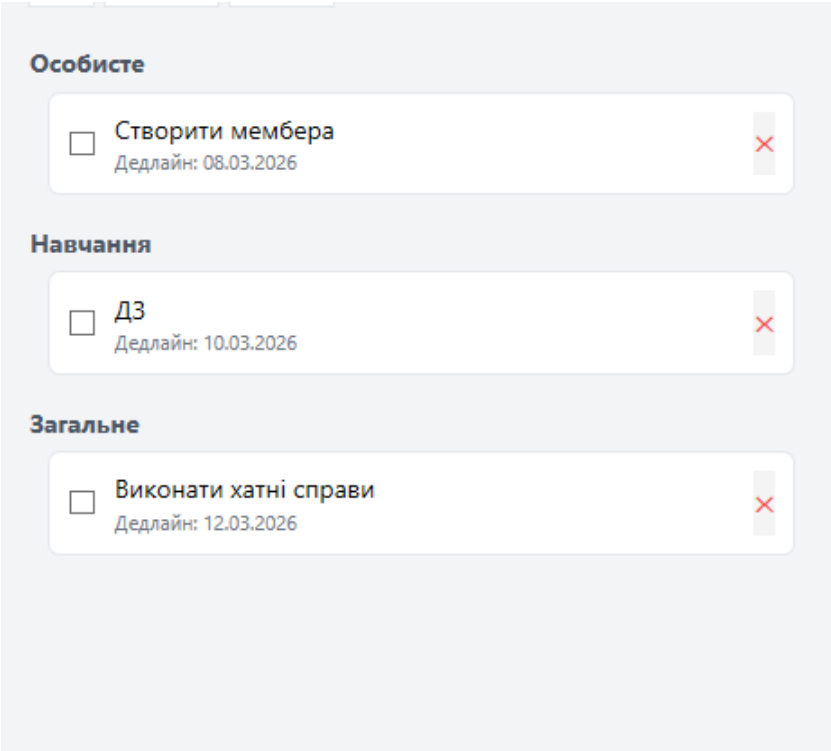


Рис. 3.3. Відображення списку задач та елементи фільтрації

Права частина вікна відведена під головний інструмент тайм-менеджменту — панель трекінгу часу. Коли користувач обирає певну задачу зі списку, клікнувши по ній мишею, її деталі підвантажуються в цю зону. Тут розміщено великий електронний таймер у форматі ГГ:ХХ:СС та елементи керування відліком: кнопки «Старт» та «Стоп» (Рис. 3.4).

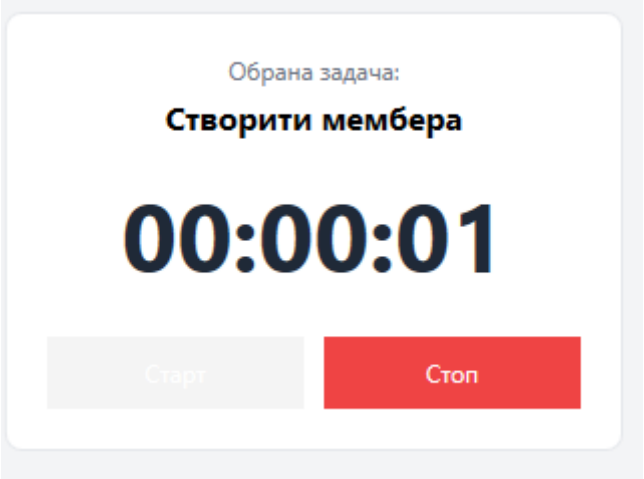


Рис. 3.4. Панель таймера та управління часом виконання

Сценарій типової роботи з додатком виглядає наступним чином: користувач додає необхідні задачі на день та сортує їх за пріоритетом. Далі він обирає найважливішу задачу, натискає кнопку «Старт» і починає роботу. Таймер у реальному часі відображає кількість витраченого часу. У разі необхідності зробити перерву, натискається кнопка «Стоп». Після повного завершення роботи над завданням, користувач відмічає чекбокс на картці задачі — вона закреслюється і переходить у статус виконаних, звільняючи робочий простір для наступних цілей.

### 3.2 Тестування роботи програмного забезпечення

Для забезпечення стабільної роботи додатку та уникнення запису некоректних даних у збережений JSON-файл, було реалізовано механізм валідації вхідних даних. Основна увага при тестуванні приділялася процесу створення нової задачі та логіці роботи таймера, оскільки це ключові функції системи.

За допомогою методів перевірки у MainViewModel, форма додавання суворо валідується. Наприклад, система не дозволить створити завдання без назви або з назвою, яка складається виключно з пробілів. Це реалізовано через перевірку стану команди Command CanExecute.

```
public class MainViewModel : INotifyPropertyChanged
{
    private bool CanAddTask(object parameter) =>
!string.IsNullOrWhiteSpace(NewTaskTitle);

    private void AddTask(object parameter)
    {
        Tasks.Add(new TaskItem
        {
            Title = NewTaskTitle,
            Category = SelectedCategory,
            Deadline = NewTaskDeadline
        });
        NewTaskTitle = string.Empty;
        NewTaskDeadline = DateTime.Today;
    }
  }
```

Завдяки цьому коду, інтерфейс WPF автоматично робить кнопку «Додати» неактивною сірою, поки користувач не введе валідний текст. Це унеможливлює додавання пустих елементів у список (Рис. 3.5).

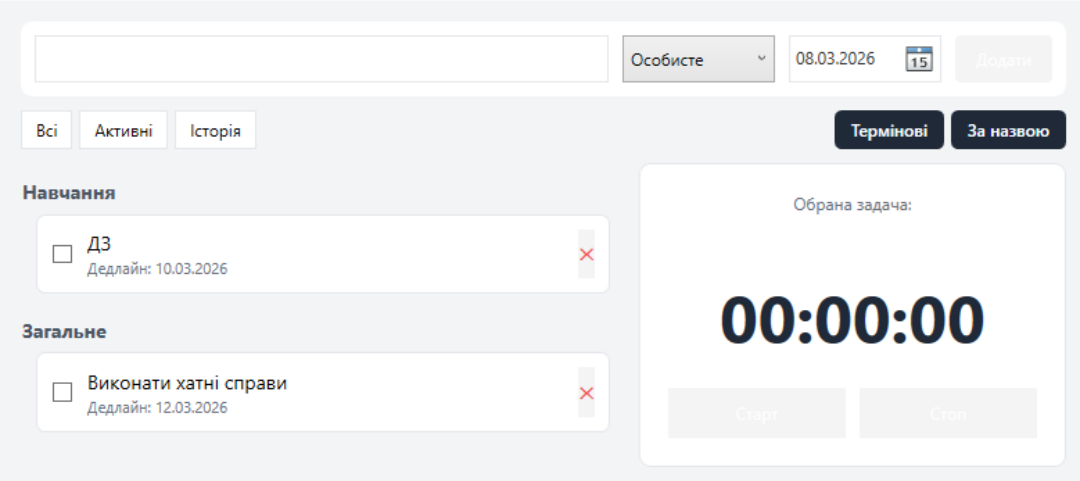


Рис. 3.5. Блокування кнопки при порожніх вхідних даних

Крім валідації полів, було проведено функціональне тестування логіки відліку часу. Під час ручного тестування перевірялися такі сценарії Test Cases:

1. **Запуск та зупинка:** Таймер коректно починає відлік при натисканні «Старт» і зупиняється на поточному значенні при натисканні «Стоп».
2. **Перемикання між задачами:** Якщо користувач обирає іншу задачу в списку під час працюючого таймера, відлік для попередньої задачі автоматично

зупиняється. Це гарантує, що час не буде помилково зарахований іншому завданню.

3. **Завершення задачі:** Якщо відмітити чекбокс «Виконано» IsCompleted = true під час активного таймера, відлік примусово припиняється, оскільки задача переходить у статус закритої (Рис. 3.6).

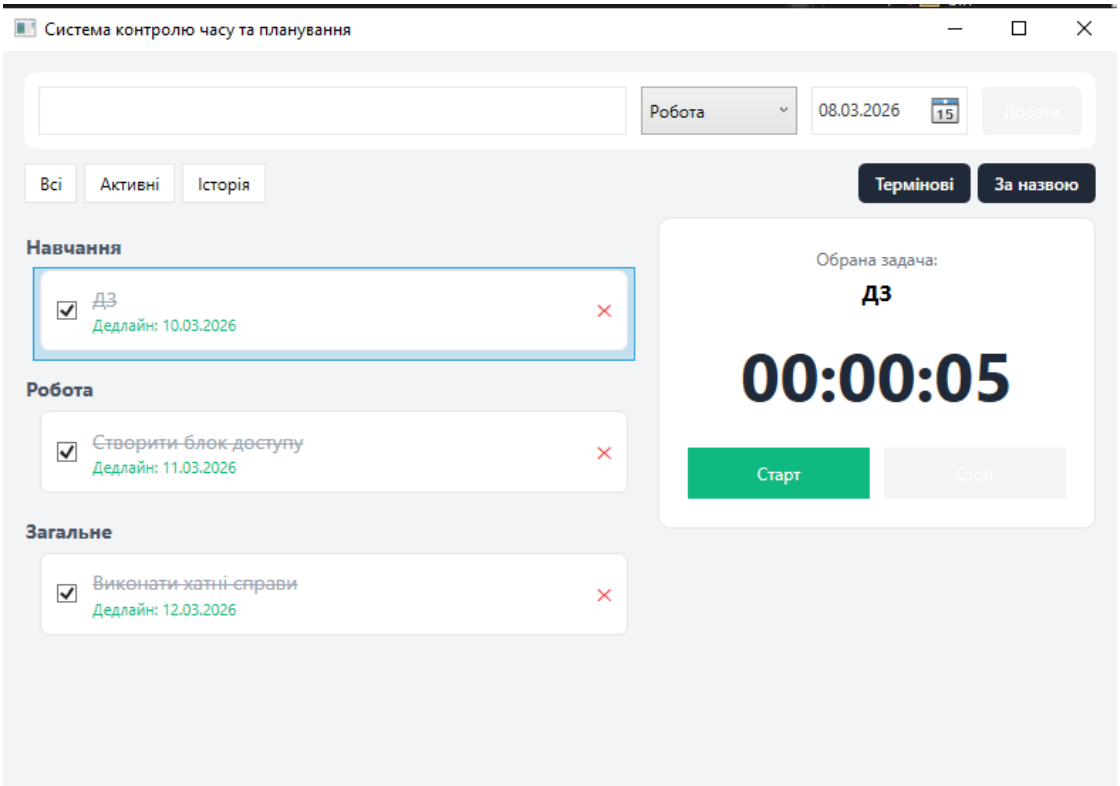


Рис. 3.6. Фіксація часу для виконаної задачі

**Висновок:** У третьому розділі було детально описано роботу з розробленим програмним додатком. Розглянуто зручний користувацький інтерфейс, який відповідає сучасним вимогам до інструментів тайм-менеджменту та працює за принципом єдиного вікна. Проведено тестування основних модулів системи, зокрема механізмів валідації вхідних даних та логіки роботи секундоміра. Результати тестування підтверджують, що програма працює стабільно, коректно обробляє дії користувача, миттєво реагує на зміни та надійно захищена від введення пустих даних.

# ВИСНОВКИ

Отже, в результаті виконання курсової роботи було розроблено настільну програмну систему контролю часу та планування задач. Процес створення продукту відбувався в кілька послідовних етапів, від аналізу предметної області до фінального тестування.

На основі проведеного аналізу існуючих рішень було вирішено зосередитись на розробці гібридного інструменту, що гармонійно поєднує функції планувальника To-Do та трекера часу. Основна увага приділялась створенню зручної та інтуїтивно зрозумілої взаємодії користувача з програмою за принципом «одного вікна». Використання архітектурного патерну MVVM у поєднанні з платформою WPF забезпечило високу гнучкість, масштабованість та чистоту коду.

Проектування загального алгоритму роботи програми базувалося на ефективному управлінні станами відображення за допомогою механізмів Data Binding та CollectionView. Це забезпечило чітке розділення візуальної частини View та бізнес-логіки ViewModel, що дозволяє легко підтримувати та розширювати інтерфейс без втручання в основні алгоритми.

Управління всіма ключовими процесами здійснювалося через головний клас MainViewModel, який об'єднує роботу з колекціями задач, алгоритмами сортування за терміновістю та сервісом відліку часу на базі DispatcherTimer. Розробка функціональних моделей включала створення класу TaskItem. Всі моделі даних наслідують BaseViewModel та використовують інтерфейс INotifyPropertyChanged для динамічного відстеження змін, забезпечуючи миттєву та інтерактивну реакцію інтерфейсу на дії користувача чи спрацювання таймера.

Для збереження даних користувача замість громіздких реляційних баз даних було реалізовано автономне локальне сховище на основі формату JSON. Розроблений сервіс JsonDataService забезпечує швидке та надійне збереження всього списку завдань, їхніх статусів, дедлайнів та накопиченого часу між сеансами роботи додатку.

Загалом, проектування та розробка програми дозволили створити добре структурований, розширюваний та стабільний продукт, що сприяє ефективній

організації особистого часу. Програма була успішно протестована на предмет валідації вхідних даних, коректності роботи таймера та механізмів збереження, що повністю підтвердило її функціональність та відповідність поставленим цілям.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ


1. EPAM Campus. Основні принципи ООП [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://training.epam.ua/ua/blog/275
2. Microsoft Learn. Model-View-ViewModel (MVVM) [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm
3. Microsoft Learn. Windows Presentation Foundation documentation. [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/?view=netdesktop-8.0
4. Microsoft Learn. System.Text.Json documentation [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json-overview
5. Microsoft Learn. DispatcherTimer Class [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://learn.microsoft.com/en-us/dotnet/api/system.windows.threading.dispatchertimer
6. Microsoft Learn. CollectionViewSource Class [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://learn.microsoft.com/en-us/dotnet/api/system.windows.data.collectionviewsource
7. Medium. Time Management & Productivity for Software Engineers [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://medium.com/swlh/time-management-productivity-for-software-engineers-5a3b9062eb1
8. YouTube. C# WPF Tutorial [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу:

https://www.youtube.com/playlist?list=PLih2KERbY1HHOOJ2C6FOrVXIwg4AZ-hk1

9. YouTube. Implementing INotifyDataErrorInfo (View Model Data Validation) - EASY WPF (.NET CORE) [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://www.youtube.com/watch?v=JZth6r2UXLU
10. YouTube. WPF MVVM - To-Do List App Tutorial [Електронний ресурс]: [Веб-сайт]. – Електронні дані. – Режим доступу: https://www.youtube.com/watch?v=W3e_U0yL8gI

# ДОДАТКИ

## Файл TaskItem.cs

```
using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace WpfApp1.Models
{
    public class TaskItem : INotifyPropertyChanged
    {
        private string _title = string.Empty;
        private string _category = "Загальне";
        private bool _isCompleted = false;
        private int _timeSpentSeconds = 0;
        private DateTime? _deadline;

        public string Id { get; set; } = Guid.NewGuid().ToString();

        public string Title
        {
            get => _title;
            set { _title = value; OnPropertyChanged(); }
        }

        public string Category
        {
            get => _category;
            set { _category = value; OnPropertyChanged(); }
        }

        public bool IsCompleted
        {
            get => _isCompleted;
            set
            {
                _isCompleted = value;
                OnPropertyChanged();
                OnPropertyChanged(nameof(IsOverdue));
            }
        }

        public DateTime? Deadline
        {
            get => _deadline;
            set
            {
                _deadline = value;
                OnPropertyChanged();
                OnPropertyChanged(nameof(IsOverdue));
            }
        }

        public bool IsOverdue => Deadline.HasValue && Deadline.Value.Date <
DateTime.Now.Date && !IsCompleted;

        public DateTime CreatedAt { get; set; } = DateTime.Now;

        public int TimeSpentSeconds
        {
            get => _timeSpentSeconds;
            set
            {
                _timeSpentSeconds = value;
                OnPropertyChanged();
```

```
                OnPropertyChanged(nameof(FormattedTime));
            }
        }

        public string FormattedTime =>
TimeSpan.FromSeconds(TimeSpentSeconds).ToString(@"hh\:mm\:ss");

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
   }
```

## Файл MainViewModel.cs

```
using System;
using System.Collections.ObjectModel;
using System.Collections.Specialized;
using System.ComponentModel;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Windows.Data;
using System.Windows.Input;
using System.Windows.Threading;
using WpfApp1.Models;
using WpfApp1.Services;

namespace WpfApp1.ViewModels
{
    public class MainViewModel : INotifyPropertyChanged
    {
        private readonly JsonDataService _dataService;
        private string _newTaskTitle = string.Empty;
        private string _selectedCategory;
        private DateTime? _newTaskDeadline = DateTime.Today;
        private TaskItem _selectedTask;

        private DispatcherTimer _timer;
        private bool _isTimerRunning;
        private string _currentFilter = "All";

        // Поля статистики
        private int _totalTasksCount;
        private int _completedTasksCount;
        private string _totalTimeSpent = "00:00:00";

        public ObservableCollection<TaskItem> Tasks { get; set; }
        public ObservableCollection<string> Categories { get; set; }
        public ICollectionView TasksView { get; }

        // Властивості для статистики
        public int TotalTasksCount
        {
            get => _totalTasksCount;
            set { _totalTasksCount = value; OnPropertyChanged(); }
        }

        public int CompletedTasksCount
        {
            get => _completedTasksCount;
```

```
            set { _completedTasksCount = value; OnPropertyChanged(); }
        }

        public string TotalTimeSpent
        {
            get => _totalTimeSpent;
            set { _totalTimeSpent = value; OnPropertyChanged(); }
        }

        public string NewTaskTitle
        {
            get => _newTaskTitle;
            set { _newTaskTitle = value; OnPropertyChanged(); }
        }

        public string SelectedCategory
        {
            get => _selectedCategory;
            set { _selectedCategory = value; OnPropertyChanged(); }
        }

        public DateTime? NewTaskDeadline
        {
            get => _newTaskDeadline;
            set { _newTaskDeadline = value; OnPropertyChanged(); }
        }

        public TaskItem SelectedTask
        {
            get => _selectedTask;
            set
            {
                if (_isTimerRunning) StopTimer(null);
                _selectedTask = value;
                OnPropertyChanged();
            }
        }

        public bool IsTimerRunning
        {
            get => _isTimerRunning;
            set { _isTimerRunning = value; OnPropertyChanged(); }
        }

        public ICommand AddTaskCommand { get; }
        public ICommand DeleteTaskCommand { get; }
        public ICommand StartTimerCommand { get; }
        public ICommand StopTimerCommand { get; }
        public ICommand FilterCommand { get; }
        public ICommand SortCommand { get; }

        public MainViewModel()
        {
            _dataService = new JsonDataService();
            var loadedTasks = _dataService.LoadTasks();
            Tasks = new ObservableCollection<TaskItem>(loadedTasks);

            Categories = new ObservableCollection<string> { "Загальне", "Навчання",
"Робота", "Особисте" };
            SelectedCategory = Categories.First();

            TasksView = CollectionViewSource.GetDefaultView(Tasks);
            TasksView.Filter = FilterTasksCondition;
            TasksView.GroupDescriptions.Add(new PropertyGroupDescription("Category"));
```

```
            // Сортування за терміновістю
            TasksView.SortDescriptions.Add(new SortDescription("IsCompleted",
ListSortDirection.Ascending));
            TasksView.SortDescriptions.Add(new SortDescription("Deadline",
ListSortDirection.Ascending));

            Tasks.CollectionChanged += Tasks_CollectionChanged;
            foreach (var task in Tasks) task.PropertyChanged += Task_PropertyChanged;

            UpdateStatistics();

            _timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(1) };
            _timer.Tick += Timer_Tick;

            AddTaskCommand = new RelayCommand(AddTask, CanAddTask);
            DeleteTaskCommand = new RelayCommand(DeleteTask, CanDeleteTask);
            StartTimerCommand = new RelayCommand(StartTimer, CanStartTimer);
            StopTimerCommand = new RelayCommand(StopTimer, CanStopTimer);
            FilterCommand = new RelayCommand(ChangeFilter);
            SortCommand = new RelayCommand(ApplySort);
        }

        private void UpdateStatistics()
        {
            TotalTasksCount = Tasks.Count;
            CompletedTasksCount = Tasks.Count(t => t.IsCompleted);

            long totalSeconds = Tasks.Sum(t => t.TimeSpentSeconds);
            TotalTimeSpent = TimeSpan.FromSeconds(totalSeconds).ToString(@"hh\:mm\:ss");
        }

        // Метод сортування
        private void ApplySort(object parameter)
        {
            TasksView.SortDescriptions.Clear();

            if (parameter is string sortType && sortType == "Urgency")
            {

                TasksView.SortDescriptions.Add(new SortDescription("IsCompleted",
ListSortDirection.Ascending));
                TasksView.SortDescriptions.Add(new SortDescription("Deadline",
ListSortDirection.Ascending));
            }
            else
            {

                TasksView.SortDescriptions.Add(new SortDescription("Category",
ListSortDirection.Ascending));
                TasksView.SortDescriptions.Add(new SortDescription("Title",
ListSortDirection.Ascending));
            }

            TasksView.Refresh();
        }

        private bool FilterTasksCondition(object obj)
        {
            if (obj is TaskItem task)
            {
                return _currentFilter switch
                {
                    "Active" => !task.IsCompleted,
```

```
                    "Completed" => task.IsCompleted,
                    _ => true
                };
            }
            return false;
        }

        private void ChangeFilter(object parameter)
        {
            if (parameter is string filter)
            {
                _currentFilter = filter;
                TasksView.Refresh();
            }
        }

        private void Timer_Tick(object sender, EventArgs e)
        {
            if (SelectedTask != null)
            {
                SelectedTask.TimeSpentSeconds++;
                UpdateStatistics();
            }
        }

        private bool CanStartTimer(object parameter) => SelectedTask != null &&
!IsTimerRunning;
        private void StartTimer(object parameter) { IsTimerRunning = true;
_timer.Start(); }

        private bool CanStopTimer(object parameter) => IsTimerRunning;
        private void StopTimer(object parameter) { IsTimerRunning = false; _timer.Stop();
SaveData(); }

        private void Tasks_CollectionChanged(object sender,
NotifyCollectionChangedEventArgs e)
        {
            if (e.NewItems != null) foreach (TaskItem item in e.NewItems)
item.PropertyChanged += Task_PropertyChanged;
            if (e.OldItems != null) foreach (TaskItem item in e.OldItems)
item.PropertyChanged -= Task_PropertyChanged;

            UpdateStatistics();
            SaveData();
        }

        private void Task_PropertyChanged(object sender, PropertyChangedEventArgs e)
        {
            if (e.PropertyName == nameof(TaskItem.TimeSpentSeconds) && IsTimerRunning)
            {
                return;
            }

            if (e.PropertyName == nameof(TaskItem.IsCompleted) || e.PropertyName ==
nameof(TaskItem.TimeSpentSeconds))
            {
                UpdateStatistics();
            }

            if (e.PropertyName == nameof(TaskItem.IsCompleted)) TasksView.Refresh();

            SaveData();
        }
```

```
        private bool CanAddTask(object parameter) =>
!string.IsNullOrWhiteSpace(NewTaskTitle);

        private void AddTask(object parameter)
        {
            Tasks.Add(new TaskItem
            {
                Title = NewTaskTitle,
                Category = SelectedCategory,
                Deadline = NewTaskDeadline
            });
            NewTaskTitle = string.Empty;
            NewTaskDeadline = DateTime.Today;
        }

        private bool CanDeleteTask(object parameter) => SelectedTask != null;
        private void DeleteTask(object parameter) { if (SelectedTask != null)
Tasks.Remove(SelectedTask); }

        private void SaveData() => _dataService.SaveTasks(Tasks.ToList());

        public event PropertyChangedEventHandler PropertyChanged;
        protected void OnPropertyChanged([CallerMemberName] string name = null) =>
PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
    }
      }
```

## Файл JsonDataService.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.Json;
using WpfApp1.Models;

namespace WpfApp1.Services
{
    public class JsonDataService
    {
        private readonly string _filePath;

        public JsonDataService(string fileName = "tasks.json")
        {
            _filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, fileName);
        }

        public void SaveTasks(IEnumerable<TaskItem> tasks)
        {
            var options = new JsonSerializerOptions { WriteIndented = true };
            string json = JsonSerializer.Serialize(tasks, options);
            File.WriteAllText(_filePath, json);
        }

        public IEnumerable<TaskItem> LoadTasks()
        {
            if (!File.Exists(_filePath))
            {
                return new List<TaskItem>();
            }

            string json = File.ReadAllText(_filePath);
```

```
            return JsonSerializer.Deserialize<IEnumerable<TaskItem>>(json) ?? new
List<TaskItem>();
        }
    }
      }
```

## Файл MainWindow.xaml

```
      <Window x:Class="WpfApp1.MainWindow"
              xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
              xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
              Title="Система контролю часу та планування" Height="600" Width="800"
Background="#FFF3F4F6">

          <Grid Margin="15">
              <Grid.RowDefinitions>
                  <RowDefinition Height="Auto"/>
                  <RowDefinition Height="Auto"/>
                  <RowDefinition Height="*"/>
                  <RowDefinition Height="Auto"/>
              </Grid.RowDefinitions>

              <Border Grid.Row="0" Background="White" Padding="10" CornerRadius="8"
Margin="0,0,0,10">
                  <Grid>
                      <Grid.ColumnDefinitions>
                          <ColumnDefinition Width="*"/>
                          <ColumnDefinition Width="Auto"/>
                          <ColumnDefinition Width="Auto"/>
                          <ColumnDefinition Width="Auto"/>
                      </Grid.ColumnDefinitions>

                      <TextBox Grid.Column="0" Text="{Binding NewTaskTitle,
UpdateSourceTrigger=PropertyChanged}"
                               Margin="0,0,10,0" VerticalAlignment="Center" Padding="8"
BorderBrush="#E5E7EB" BorderThickness="1"
                               ToolTip="Введіть назву задачі..."/>

                      <ComboBox Grid.Column="1" ItemsSource="{Binding Categories}"
SelectedItem="{Binding SelectedCategory}"
                                Margin="0,0,10,0" Width="110"
VerticalContentAlignment="Center" BorderBrush="#E5E7EB"/>

                      <DatePicker Grid.Column="2" SelectedDate="{Binding
NewTaskDeadline}"
```

```
                                Margin="0,0,10,0"                          Width="110"
VerticalContentAlignment="Center" BorderBrush="#E5E7EB"
                                ToolTip="Оберіть дедлайн"/>

                    <Button    Grid.Column="3"    Content="Додати"    Command="{Binding
AddTaskCommand}"
                            Padding="15,8"    Background="#3B82F6"    Foreground="White"
BorderThickness="0" Cursor="Hand">
                        <Button.Resources>
                            <Style TargetType="Border">
                                <Setter Property="CornerRadius" Value="4"/>
                            </Style>
                        </Button.Resources>
                    </Button>
                </Grid>
            </Border>

            <Grid Grid.Row="1" Margin="0,0,0,10">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="Auto"/>
                </Grid.ColumnDefinitions>

                <StackPanel Orientation="Horizontal">
                    <Button     Content="Всі"     Command="{Binding     FilterCommand}"
CommandParameter="All"      Margin="0,0,5,0"      Padding="10,5"      Background="White"
BorderBrush="#E5E7EB"/>
                    <Button    Content="Активні"    Command="{Binding    FilterCommand}"
CommandParameter="Active"     Margin="0,0,5,0"     Padding="10,5"     Background="White"
BorderBrush="#E5E7EB"/>
                    <Button    Content="Історія"    Command="{Binding    FilterCommand}"
CommandParameter="Completed" Padding="10,5" Background="White" BorderBrush="#E5E7EB"/>
                </StackPanel>

                <StackPanel          Grid.Column="1"          Orientation="Horizontal"
HorizontalAlignment="Right">

                    <Button Command="{Binding SortCommand}" CommandParameter="Urgency"
                            Margin="0,0,5,0"    Padding="12,6"    Background="#1F2937"
BorderThickness="0" Cursor="Hand"
                            ToolTip="Спочатку прострочені та найближчі">
                        <Button.Resources>
                            <Style TargetType="Border">
```

```
                                <Setter Property="CornerRadius" Value="5"/>
                            </Style>
                        </Button.Resources>
                        <TextBlock         Text="Термінові"         Foreground="White"
FontWeight="SemiBold"/>
                    </Button>

                    <Button Command="{Binding SortCommand}" CommandParameter="Title"
                            Padding="12,6"   Background="#1F2937"   BorderThickness="0"
Cursor="Hand"
                            ToolTip="Сортування за алфавітом">
                        <Button.Resources>
                            <Style TargetType="Border">
                                <Setter Property="CornerRadius" Value="5"/>
                            </Style>
                        </Button.Resources>
                        <TextBlock      Text="За      назвою"      Foreground="White"
FontWeight="SemiBold"/>
                    </Button>

                </StackPanel>
            </Grid>

            <Grid Grid.Row="2">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="2*"/>
                    <ColumnDefinition Width="1.5*"/>
                </Grid.ColumnDefinitions>

                <ListBox     Grid.Column="0"     ItemsSource="{Binding     TasksView}"
SelectedItem="{Binding SelectedTask}"
                         HorizontalContentAlignment="Stretch"      BorderThickness="0"
Background="Transparent"
                         ScrollViewer.HorizontalScrollBarVisibility="Disabled">

                    <ListBox.GroupStyle>
                        <GroupStyle>
                            <GroupStyle.HeaderTemplate>
                                <DataTemplate>
                                    <TextBlock Text="{Binding Name}" FontWeight="Bold"
Foreground="#4B5563" FontSize="14" Margin="0,10,0,5"/>
                                </DataTemplate>
                            </GroupStyle.HeaderTemplate>
```

```
                        </GroupStyle>
                    </ListBox.GroupStyle>

                    <ListBox.ItemTemplate>
                        <DataTemplate>
                            <Border  Background="White"  Margin="0,0,0,5"  Padding="10"
CornerRadius="6" BorderBrush="#E5E7EB" BorderThickness="1">
                                <Grid>
                                    <Grid.ColumnDefinitions>
                                        <ColumnDefinition Width="Auto"/>
                                        <ColumnDefinition Width="*"/>
                                        <ColumnDefinition Width="Auto"/>
                                        <ColumnDefinition Width="Auto"/>
                                    </Grid.ColumnDefinitions>

                                    <CheckBox    Grid.Column="0"    IsChecked="{Binding
IsCompleted, UpdateSourceTrigger=PropertyChanged}"
                                                 VerticalAlignment="Center"
Margin="0,0,10,0"/>

                                    <StackPanel                        Grid.Column="1"
VerticalAlignment="Center">
                                        <TextBlock Text="{Binding Title}" FontSize="14"
TextWrapping="Wrap" Margin="0,0,0,2">
                                            <TextBlock.Style>
                                                <Style TargetType="TextBlock">
                                                    <Style.Triggers>
                                                        <DataTrigger  Binding="{Binding
IsCompleted}" Value="True">
                                                            <Setter
Property="TextDecorations" Value="Strikethrough"/>
                                                            <Setter
Property="Foreground" Value="#9CA3AF"/>
                                                        </DataTrigger>
                                                    </Style.Triggers>
                                                </Style>
                                            </TextBlock.Style>
                                        </TextBlock>

                                        <TextBlock      Text="{Binding      Deadline,
StringFormat='Дедлайн: {0:dd.MM.yyyy}'}" FontSize="11">
                                            <TextBlock.Style>
                                                <Style TargetType="TextBlock">
```

```
                                                        <Setter      Property="Foreground"
Value="#6B7280"/>
                                                        <Style.Triggers>
                                                            <DataTrigger  Binding="{Binding
IsOverdue}" Value="True">
                                                                <Setter
Property="Foreground" Value="#EF4444"/>
                                                                <Setter
Property="FontWeight" Value="Bold"/>
                                                                <Setter     Property="Text"
Value="{Binding Deadline, StringFormat='ПРОСТРОЧЕНО: {0:dd.MM.yyyy}'}"/>
                                                            </DataTrigger>
                                                            <DataTrigger  Binding="{Binding
IsCompleted}" Value="True">
                                                                <Setter
Property="Foreground" Value="#10B981"/>
                                                            </DataTrigger>
                                                        </Style.Triggers>
                                                    </Style>
                                                </TextBlock.Style>
                                            </TextBlock>
                                        </StackPanel>

                                        <Button        Grid.Column="3"        Content="×"
Command="{Binding      DataContext.DeleteTaskCommand,      RelativeSource={RelativeSource
AncestorType=Window}}"
                                                Background="Transparent"
BorderThickness="0"       Foreground="#EF4444"       FontWeight="Bold"       Cursor="Hand"
Margin="5,0,0,0"/>
                                    </Grid>
                                </Border>
                            </DataTemplate>
                        </ListBox.ItemTemplate>
                    </ListBox>

                    <Border      Grid.Column="1"      Margin="15,0,0,0"      Padding="20"
Background="White"     CornerRadius="8"     BorderBrush="#E5E7EB"     BorderThickness="1"
VerticalAlignment="Top">
                        <StackPanel VerticalAlignment="Center">
                            <TextBlock    Text="Обрана    задача:"    Foreground="#6B7280"
FontSize="12" HorizontalAlignment="Center"/>
                            <TextBlock  Text="{Binding  SelectedTask.Title}"  FontSize="16"
FontWeight="Bold" HorizontalAlignment="Center"
```

```
                                    Margin="0,5,0,15"               TextWrapping="Wrap"
TextAlignment="Center"/>

                        <TextBlock      Text="{Binding      SelectedTask.FormattedTime,
TargetNullValue='00:00:00', FallbackValue='00:00:00'}"
                                    FontSize="48"                       FontWeight="Bold"
HorizontalAlignment="Center" Margin="0,0,0,20" Foreground="#1F2937"/>

                        <Grid>
                            <Grid.ColumnDefinitions>
                                <ColumnDefinition Width="*"/>
                                <ColumnDefinition Width="*"/>
                            </Grid.ColumnDefinitions>
                            <Button  Grid.Column="0"  Content="Старт"  Command="{Binding
StartTimerCommand}"
                                     Margin="0,0,5,0"  Padding="10"  Background="#10B981"
Foreground="White" BorderThickness="0" Cursor="Hand"/>
                            <Button  Grid.Column="1"  Content="Стоп"  Command="{Binding
StopTimerCommand}"
                                     Margin="5,0,0,0"  Padding="10"  Background="#EF4444"
Foreground="White" BorderThickness="0" Cursor="Hand"/>
                        </Grid>
                    </StackPanel>
                </Border>
            </Grid>

            <Border  Grid.Row="3"  Background="#1F2937"  Padding="10,5"  CornerRadius="5"
Margin="0,10,0,0">
                <Grid>
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition Width="*"/>
                        <ColumnDefinition Width="Auto"/>
                    </Grid.ColumnDefinitions>

                    <TextBlock Grid.Column="0" Foreground="White" Margin="0,0,15,0">
                        <Run Text="Всього задач: " FontWeight="SemiBold"/>
                        <Run Text="{Binding TotalTasksCount}"/>
                    </TextBlock>

                    <TextBlock Grid.Column="1" Foreground="White" Margin="0,0,15,0">
                        <Run Text="Виконано: " FontWeight="SemiBold"/>
```

```
                        <Run Text="{Binding CompletedTasksCount}"/>
                    </TextBlock>

                    <TextBlock Grid.Column="3" Foreground="#FCD34D" FontWeight="Bold">
                        <Run Text="Загальний час: "/>
                        <Run Text="{Binding TotalTimeSpent}"/>
                    </TextBlock>
                </Grid>
            </Border>
        </Grid>
    </Window>
```

## Файл MainWindow.xaml.cs

```
using System.Windows;
using WpfApp1.ViewModels;

namespace WpfApp1
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            DataContext = new MainViewModel();
        }
    }
    }
```

## Файл RelayCommand.cs

```
using System;
using System.Windows.Input;

namespace WpfApp1.ViewModels
{
    public class RelayCommand : ICommand
    {
        private readonly Action<object> _execute;
        private readonly Predicate<object> _canExecute;

        public RelayCommand(Action<object> execute, Predicate<object> canExecute = null)
        {
            _execute = execute ?? throw new ArgumentNullException(nameof(execute));
            _canExecute = canExecute;
        }

        public event EventHandler CanExecuteChanged
        {
            add { CommandManager.RequerySuggested += value; }
            remove { CommandManager.RequerySuggested -= value; }
        }

        public bool CanExecute(object parameter) => _canExecute == null ||
_canExecute(parameter);
```

```
        public void Execute(object parameter) => _execute(parameter);
    }
    }
```